

《Meow準快跑》決賽簡報講稿

對應簡報：[Deliverables/最終作品繳交再想-Meow準快跑.pptx](#) (20 頁，2026-08-13 版)

形式：一人上台 | 總長 15 分鐘 | 含約 2 分鐘現場 Demo

主講目標時間 **13 分 15 秒**，保留 1 分 45 秒緩衝——現場 Demo 會載入、會等網路，緩衝不能省。

⚠️ 上台前必須先改的三件事

這三件都會被評審當場看到，且都不是大工程。

1. Slide 18 的數字和 Slide 19 自相矛盾

同一份簡報裡兩頁講同一組統計，數字卻不一樣。評審翻回去對照就會發現。

項目	Slide 18 現寫	Slide 19 現寫	repo 實測 (2026-08-13)	建議改成
任務卡	27	28	28	28
人類決策	25 (D-001-D-025)	26	D-001 ~ D-026	26 (D-001-D-026)
Git commits	120+	140+	150 (開發期至 07-16 為 136)	130+ (Slide 18 指開發期) / Slide 19 維持 140+
GDScript	5,500+ (36 份)	7,000+	7,082 行 · 38 檔	7,000+ (38 份)
AI 提問	12	12	12	不用改 ✓
期程	~3 週 · 06-27 → 07-16	—	首 commit 06-27、任務 28 落在 07-16	不用改 ✓

「~3 週」是正確的，不要改——首個 commit 2026-06-27，最後一張任務卡 2026-07-16 完成，確實約三週。

2. 遊戲裡顯示的名字不是《Meow準快跑》

`Data/ui_text.json` 的 `title_game_name` 目前是「勇者撤離戰」，Demo 一開畫面評審就會看到跟簡報封面不同的名字。

而這件事剛好可以變成加分：它就在 Slide 7 講的那份 `ui_text.json` 裡，改一個 key、不動任何程式。建議直接改掉，然後在 Slide 7 講稿加一句「這頁講的東西，我們昨天才用它改掉遊戲標題」。

```
# Data/ui_text.json 第 8 行: "title_game_name": "勇者撤離戰" → "Meow準快跑"
```

改完要重新 export H5 並部署，請留時間。**如果來不及改，Demo 時務必主動說一句：**「遊戲內標題是開發期的暫定名，正式名稱是 Meow準快跑」——被評審先問到比自己先講傷得多。

3. Slide 8 和 Slide 10 是同一件事講兩次

兩頁都是同樣的八個步驟（發想 → 拆規格 → Q→D → 單一真實來源 → 平行執行 → 自動驗證 → 人類驗收 → 上線）。Slide 8 是彩色流程圖、Slide 10 是八宮格。

本講稿的處理方式：**保留兩頁，但給它們不同的任務**——Slide 8 只花 25 秒講「顏色 = 誰負責」，Slide 10 才是落錘的那頁。

更好的選擇是直接砍掉 Slide 8（Slide 10 的標題本身就帶論點，而且用桃紅色標出了③和⑦兩個人類把關點）。砍掉可多出 25 秒還給 Demo 緩衝。要砍的話，本講稿 Slide 8 段整段跳過即可，其餘不受影響。

時間配置

時間	長度	頁	內容
0:00–0:35	0:35	1	封面：雙重交付
0:35–0:50	0:15	2	Overview
0:50–1:35	0:45	3	遊戲定位
1:35–2:20	0:45	4	市場痛點與我們的機會
2:20–3:10	0:50	5	AI 工具總覽
3:10–3:45	0:35	6	人類 / AI 分工
3:45–4:30	0:45	7	數值不寫死 (JSON 治理)
4:30–4:55	0:25	8	程式開發流程 (快速帶過)
4:55–5:40	0:45	9	美術製作流程
5:40–6:20	0:40	10	任務卡 = 共同介面
6:20–6:50	0:30	11	repo 實況截圖
6:50–7:40	0:50	12	# 1 失憶與漂移
7:40–8:05	0:25	13	placeholder → 正式美術對照
8:05–8:30	0:25	14	UI 迭代對照
8:30–9:15	0:45	15	# 2 工程 / 美術平行
9:15–9:35	0:20	16	agent-sprite-forge
9:35–11:35	2:00	17	現場 Demo
11:35–12:10	0:35	18	量化成果
12:10–12:55	0:45	19	可複製性收束
12:55–13:05	0:10	20	致謝
13:05–15:00	1:55	—	緩衝與 Q&A

時間吃緊時的砍法順序：① 砍 Slide 8 (-25s) ② Slide 16 縮到 10 秒 ③ Slide 13、14 併成一句話帶過。

絕對不要砍：Slide 7 (JSON)、Slide 12 (# 1 治理)、Demo。

逐頁講稿

講稿為口語逐字稿，可直接念。[] 內是動作提示，不要念出來。

Slide 1 | 封面 (0:35)

這頁要達成：讓評審在 30 秒內知道我們交了兩樣東西，不是一款遊戲。

各位評審好，我們是「最終作品繳交再想」，作品是《Meow準快跑》。

先講結論：我們交的是兩樣東西。

第一樣，是一款現在就能打開瀏覽器直接玩的 H5 遊戲——不是影片，不是原型，是已經上線、有 Google 登入、有雲端排行榜的完整產品。

第二樣，也是我們今天真正想分享的：一條可以複製到下一個專案的 **AI 遊戲開發生產線**。

[停一拍]

這款遊戲是三週做出來的，兩個人，用下班時間。等一下我會說明，為什麼我們認為速度來自流程，而不是來自哪一個 AI 工具。

轉場：接著我用四段來說明。

Slide 2 | Overview (0:15)

這頁要達成：給評審一張地圖，讓他們知道還有多久、會聽到什麼。純過場，不要停留。

今天分成四段：先講遊戲本身和我們為什麼做這個題目；第二段是 AI 工具怎麼分工；第三段是實際做的過程和踩到的坑；第四段直接玩給各位看。

Slide 3 | 遊戲定位 (0:45)

這頁要達成：十秒讓評審懂玩法，並埋下「這是有市場依據的選擇」。

《Meow準快跑》是一款以浪貓視角重新包裝的 **風險撤離型 H5 遊戲**。

玩法十秒就能懂：你投入虛擬金幣，貓咪自動去挑戰怪物。每打贏一隻，倍率往上跳、收益跟著漲；但你隨時要決定——**現在落袋為安，還是再挑戰下一隻**。輸了，這一局的收益全部歸零。

〔手指螢幕〕短局、快速回饋、每一關都要做一次風險決策。這三件事就是它的全部。

市場上這類產品叫 **Fast Game**，具體的玩法原型叫 **Crash Game**。我們不是憑感覺選這個題目的，下一頁說明。

注意：不要在這頁展開市場數據，留給下一頁。

Slide 4 | 市場痛點與我們的機會 (0:45)

這頁要達成：證明選題是研究過的，而且我們的改動是有理由的差異化，不是美術偏好。

我們沒有一開始就叫 AI 隨便生一款遊戲，而是先看這個品類。

Crash Game 這幾年很紅，機制本身非常有效——它的張力來自「再賭一次」這個念頭。但它有幾個明顯的痛點：〔照頁面點四項，每項一句，不要念完整句〕視覺題材偏博弈、休閒玩家有距離感；玩法容易只剩一條倍率曲線在跑；失敗的挫折感很強，沒有故事去接住玩家；而且是單人的，沒有社交比較，玩完就走。

〔停一拍，轉向右側〕

所以我們的機會很清楚：**保留它高張力的決策，但把包裝整個換掉**。

抽象的倍率曲線，我們換成一隻浪貓打十關怪物；冷冰冰的 crash，我們換成戰鬥敗北；玩家輸了，還有「虎爺」這個隨機救援事件會把他救回來。再加上排行榜，讓單人遊玩也有比較的對象。

一句話：**我們借的是心理機制，不是那條曲線**。

口徑守則：全程只說「虛擬金幣」，絕不說「賭」「下注贏錢」。如果評審問，回答見 Q&A 第 1 題。

Slide 5 | AI 工具總覽 (0:50)

這頁要達成：完整揭露用了什麼、誰做什麼，並且第一次點出「交接靠文件不靠聊天」。

這是我們用到的全部 AI 工具，以及各自負責什麼。我快速帶過，重點在最後一句。

ChatGPT 用在最前面的人類發想——玩法討論、機制取舍。

Claude 和 Claude Code 負責規劃：把企劃拆成規格、資料結構和任務卡，後面也做了 Sprite Sheet 和 JSON 的資產工程。

OpenAI Codex 是實際寫 Godot 程式的那一個，也負責測試。

GPT Image-2 生靜態美術，**Magnific** 生角色動畫，**agent-sprite-forge** 生 UI 貼紙和暫代素材。

Suno AI 生 BGM。

〔加重〕這裡要特別說明一件事：**音效不是 AI 生的**。十四個音效是人類自己蒐集的素材。因為音效要的是精準跟一致，人工挑比生成更快也更穩——我們沒有為了「全 AI」而硬用 AI。

〔停一拍〕

但這頁真正的重點在下面這行小字：**每個工具只做它擅長的事，交接靠 repo 文件，不靠聊天記憶。**

工具會換，這件事不會。

注意：Godot、Photoshop、After Effects、Git、Firebase 是製作工具，不是 AI，這頁沒列是對的。被問到再說明。

Slide 6 | 人類 / AI 分工 (0:35)

這頁要達成：明確回答「那人類到底做了什麼」——這是評審一定會想的問題。

那人類做什麼？

左邊這欄全部是人類：核心玩法與產品方向、審核選項並寫入定案、要不要啟用新功能、美術風格採用、敏感的登入和憑證親手操作，最後實際遊玩驗收。

右邊是 AI：把企劃拆成規格和任務卡、提選項和風險、依照已確認的文件實作、產出符合合約的素材、自動測試和截圖。

〔指最下面那行〕

這句話是我們整個專案的立場：**AI 提供槓桿，人類握住方向盤，也握住煞車。**

注意「提選項與風險，不代替拍板」這一條——AI 可以告訴我們有三條路和各自的代價，但選哪一條是人決定的，而且要寫下來。

Slide 7 | 數值不寫死 (0:45)

這頁要達成：證明 AI 交出來的是可維護的產品，不是黑箱。這是很多 AI 專案交不出來的東西。

這頁講的是 AI 交付物的**可維護性**，我認為是這個專案最實用的一頁。

問題是這樣：如果數值散在程式碼裡，那每次想微調手感——倍率高一點、事件多一點——都得再開一次 AI、再跑一次驗收。那 AI 交出來的其實是一個黑箱，只有它自己改得動。

所以我們在 `AGENTS.md` 裡訂了第六條鐵則：**倍率、機率、文案、節奏，一律不准寫死，全部讀**

Data 底下的 JSON。

〔手掃過六張卡〕

結果就是這七份 JSON、六百多行設定，變成整款遊戲的控制台。下注區間、十關倍率盤從 1.25 到 38 倍、全部的 UI 文案、戰鬥每一拍的間隔是 0.32 秒、還有音效對應表——**人類直接改數字就好，不用回頭找 AI。**

〔指第三張卡，這句要慢〕

特別看這個 **0.16**。這是等一下 Demo 各位會看到的「虎爺」事件的觸發率。我們調整它的時候，沒有動任何一行程式，也不需要重新驗收程式。

這代表遊戲上線之後，調平衡不必依賴 AI，也不必依賴當初寫它的那個人。

〔若已改掉遊戲標題，加這句〕

順帶一提，遊戲的標題文字也在這裡面——我們昨天就是改了 `ui_text.json` 一個 key，沒碰程式。

Slide 8 | 程式開發流程 (0:25，快速帶過)

這頁要達成：只做一件事——用顏色說明「誰負責哪一段」。不要重講八個步驟，下一頁才講。

這是整條程式開發流程的全貌，我用顏色說明就好。

〔手指圖例〕紅色是人類把關、深藍是 Claude 規劃、綠色是 AI 執行、紫色是治理機制。

各位可以看到，**紅色出現在頭跟尾**——方向是人定的，驗收也是人做的；中間大部分由 AI 執行。而且驗收不過會退回去，涉及方向變更的話要重新走一次治理流程。

這八個步驟的邏輯，下一頁我用另一個角度講。

Slide 9 | 美術製作流程 (0:45)

這頁要達成：說明美術是一條有工序的產線，而且刻意保留了兩個非 AI 環節。

美術這條線我們走了八個步驟，其中有兩個刻意**不用 AI**。

一開始是人決定風格和世界觀，然後寫成 Style Bible 和 prompt 規格模板——這樣生成才會穩定。接著 GPT Image-2 生靜態素材，進 **Photoshop 標準化**，這步是人做的。再用 Magnific 生角色動畫，進 **After Effects 轉序列圖**，這步也是人做的。最後由 Claude 整理成 Sprite Sheet 加 JSON，進 GitHub 管版本。

〔指下方三行，這是重點〕

為什麼中間要卡兩個人工步驟？因為我們試過**讓 AI 直接生 Sprite Sheet**，**結果逐幀不一致、比例會漂移**，根本沒辦法用。改成「生動畫影片、再轉序列圖」就穩定了。

所以設計師在這裡的角色是 **AI Art Director**：世界觀、一致性、動畫品質，這些由人判斷。

AI 放大的是產能；真正的價值是**把 AI 產物轉換成工程可以直接整合的遊戲資產**。

Slide 10 | 任務卡 = 共同介面 (0:40)

這頁要達成：落錘。整個第二段的結論在這頁。

講完工具，我要收一個結論：**工具不是流程，任務卡才是協作的共同介面。**

為什麼？因為 ChatGPT、Claude、Codex、美術 AI 彼此不共享記憶。它們之間唯一能傳遞的東西，就是寫下來的文件。

這八步就是每一張任務卡要跑完的循環。〔指桃紅色的兩格〕

請特別看被標成桃紅色的這兩格——③ Q→D 煞車 和 ⑦ 人類驗收。這兩格是整條流程裡不能自動化的地方。

第三格的意思是：AI 遇到不明確的事情**不准猜**，要停下來寫成一個問題，等人回答。第七格的意思是：程式跑得動不算完成，要人真的去玩過。

我們有二十八張任務卡，每一張都跑完這八步。

Slide 11 | repo 實況截圖 (0:30)

這頁要達成：把前面講的抽象流程，變成看得見的檔案。這頁是證據，不是說明。

這頁是實際的 repo，我讓各位看一眼就好。

左邊是規格文件，連 UI 的座標契約都寫成文字——哪一區從第幾像素到第幾像素。右邊是二十八張任務卡的清單，和其中一張打開的樣子。

〔指右側任務卡欄位〕

每一張卡都有固定欄位：**目標、必讀文件、範圍、不做什麼、實作要求、驗收方式、完成後必須回報什麼。**

「不做什麼」這欄特別重要——它是防止 AI 順手多做的煞車。

Slide 12 | #1 失憶與漂移 (0:50)

這頁要達成：說明我們解決的第一個真問題。這是整場最有含金量的一頁。

接下來講實際做的過程。我們遇到的**第一個問題不是技術問題，是協作問題**。

只靠聊天視窗跟 AI 合作，會遇到三件事：它會忘記、會誤解、方向會慢慢漂掉。今天講好的規則，明天新開一個對話就沒有了。

我們的解法是：**讓 repo 裡的文件變成人類和所有 AI 工具的共同介面**。

六份東西撐起這件事：

`AGENTS.md` 是 AI 行為的最高鐵則，每個 AI 開工前都要讀。

`Docs` 加 `Data` 的 JSON 是單一真實來源，禁止寫死。

`OPEN_QUESTIONS.md` ——AI 不確定就停下來提問，不准猜。

`DECISIONS.md` ——人類拍板的紀錄，而且是 append-only，**推翻舊決策要開新編號，不能改掉歷史**。

`Codex` 底下是任務卡，一次一張，邊界明確。

最後用 Git 讓每一步都可追溯。

〔停一拍，這句是重點〕

這樣做的結果是：**我們的事案不依賴任何一個 session 的記憶**。換一個 AI、換一台電腦、隔兩個禮拜再回來，讀完文件就能接著做。

〔加分選項：+30 秒，時間允許就講〕

這是全場最有說服力的一段，因為它承認了失敗。時間夠就一定要講。

我舉一個我們真的做錯的例子，因為這條規則就是這樣撞出來的。

早期有一張任務卡，規格上寫的是「角色放在畫面中段、中線偏下」。AI 完全照做了——但它把角色放到正中央，直接壓住下面的操作面板。AI 沒有違規，是我們的規格太模糊。

真正的問題出在修復的時候：AI 在**沒有 Godot、看不到畫面**的情況下，直接去改場景檔的像素座標。結果面板跑版、按鈕整個消失。那批修改我們全部還原了。

從這裡我們學到：**版面是視覺產物，不能用算術盲改**。所以就有了 D-006 這條決策——把「中段」這種散文改寫成具體座標，並且定死一條規則：任何版面調整，都必須由能實際跑 Godot 的人目視確認才算完成。

各位在前面看到的驗收環節，不是我們一開始就想得周到，是被 AI 的失敗撞出來的。**流程的價值就在這裡——它讓錯誤停在還原得回來的地方**。

Slide 13 | placeholder → 正式美術 (0:25)

這頁要達成：一眼證明「工程沒有等美術」。

這頁一眼就看得懂。

左邊是工程用的 placeholder——就是幾個色塊，但**完整的遊戲循環已經跑得起來**。中間是設計師的參考稿。右邊是正式素材接進去以後的樣子。

重點是：左邊那個版本，玩法、狀態機、結算、記分**全部都已經完成了**。美術是後來換上去的，換的時候程式一行都沒動。

Slide 14 | UI 迭代對照 (0:25)

這頁要達成：證明有真實的迭代，不是一次生成就交差。

這頁是實際的迭代過程。

上面是結算畫面，下面是同一個畫面的前後版本——各位可以看到危險度的呈現從小圓點改成了星星，下注面板也重排過。

這些調整都不是重做，是**改 JSON 加上換圖**。這就是前面那條「數值不寫死」規則的回報。

Slide 15 | # 2 工程與美術平行 (0:45)

這頁要達成：說明第二個真問題——兩條線節奏不同怎麼辦。

第二個挑戰是節奏問題。

正式美術和動畫做起來很慢；但程式如果要等所有素材到齊才開工，三週絕對做不完。反過來，如果 AI 生的圖沒有尺寸、檔名、透明度的規格，最後也接不進遊戲。

所以我們讓兩條線平行跑。**工程線**用 placeholder 先把完整可玩的閉環做完；**美術線**依照 Art Contract 走自己的產線。

兩條線在這裡合流：〔指中間〕程式只認 `Assets/final` 和 placeholder 這兩個入口，**正式圖用同樣的檔名放進去就自動生效**，缺檔就 fallback，遊戲不會壞掉。

〔指下面第二行〕

還有一條規則我想特別講：**美術規格衝突的時候，不硬改**。走提案、人類裁示、然後把合約升版——我們的 Art Contract 從 1.0 一路升到 1.5，每一次升版都有紀錄。

Slide 16 | agent-sprite-forge (0:20)

⚠ 這頁最容易被誤會成「這是我們做的」。務必講清楚它是別人的開源專案。

這裡補充一個工具。`agent-sprite-forge` 是 GitHub 上 **別人開源的 Codex skill**，作者是 0x0funky，我們是使用者。

它做的事情跟我們前面講的美術產線是同一個思路——用 agent 先定資產規格，生成之後再用腳本後處理成引擎可以直接用的素材。我們拿它來生 UI 貼紙、icon 和暫代圖。

放這頁是想說明：**這套「規格先行、生成之後必須經過確定性後處理」的作法，不是只有我們這樣想**，開源社群已經在往同一個方向走了。

Slide 17 | 現場 Demo (2:00)

〔切到瀏覽器。事前已登入、停在下注畫面。〕

好，直接玩給各位看。

操作腳本

1. 投入金幣、開始挑戰

投入虛擬金幣，貓咪自動去打第一隻怪物。〔等戰鬥〕過關了，倍率往上跳，目前收益也跟著漲。

2. 第一次決策——問評審

現在有兩個選擇：帶著這些收益撤退，或者挑戰下一隻。

〔看向評審，停 2 秒〕各位覺得呢？

〔不管回應〕這個猶豫，就是整款遊戲的核心。

3. 再跑一到兩關（視時間）

4. 宣告 Demo 模式，然後按 F4

接下來我要展示一個低機率事件。它的自然觸發率是 16%，為了不讓各位等，我開一個專用的展示模式——**這不是假裝成隨機，是我們內建的 demo 開關。**

〔**先按 F4，再按「開始挑戰」**〕

⚠ F4 是「為下一次挑戰預備虎爺」，不是立刻觸發，而且畫面上完全沒有回饋。

順序按反 = 虎爺不會出現。彩排時務必確認這個順序。

5. 虎爺登場

這就是「虎爺」。玩家本來這關要輸了，虎爺會強制逆轉——各位聽一下，**BGM 也換了**——然後把整局的收益倍率翻倍。

這是我們拿來接住玩家挫折感的設計，回應的就是前面講的第三個市場痛點。

6. 撤退結算

撤退，結算。〔指畫面〕這裡會告訴你目前的排名、最高紀錄，還有一行「若再過一關可得多少」——這是刻意留的鉤子。

7. 收尾一句

補充一點：**即使登入失敗或斷網，遊戲會自動退回本地記分，主流程不會中斷**。我們沒有讓核心玩法被雲端功能綁死。

現場意外應變

狀況	怎麼講
虎爺在按 F4 前就自己跳出來 (16% , 跑 2-3 關很有機會)	不要慌也不要裝作是安排好的：「它自己出來了。自然觸發率是 16% , 各位運氣不錯——這就是玩家實際會遇到的版本。」然後跳過第 4 步直接講第 5 步。 這比 demo 模式更有說服力 。
按了 F4 虎爺沒出現	多半是順序按反。不要在台上反覆按，把這局打完照常講解，下一局再補；或切預錄影片。
網路或登入失敗	這正好是第 7 點：「各位剛好看到 fallback 了」，現場展示本地記分仍然運作。
遊戲標題還是「勇者撤離戰」	主動說：「遊戲內標題是開發期暫定名，正式名稱是 Meow 準快跑。」不要等評審問。

〔切回簡報〕

剛才看到的每一個功能——虎爺、排行榜、登入、fallback——在 repo 裡都能追到它的規格、決策、任務卡、實作和驗收紀錄。

Slide 18 | 量化成果 (0:35)

⚠ 講之前確認這頁數字已按本文開頭的表格更新，否則會跟 Slide 19 打架。

這頁要達成：把「可追溯」變成數字。快速念，不要逐項解釋。

一些數字，每一個都有 repo 證據可以查。

遊戲本體約三週完成，兩個人，下班時間。

一百三十多個 commit，每一步都可追溯。

二十八張任務卡全部完成，而且都有 Git 紀錄。

二十六筆人類決策，編號 D-001 到 D-026，可以查核。

十二次 AI 主動暫停提問，全部經過人類裁示。

七千多行 GDScript，分成三十八個檔案。

〔指最下面〕

而且它是已經上線可以玩的——Google 登入、Firebase 排行榜、七份 JSON 資料驅動，網址就在這裡。

Slide 19 | 可複製性收束 (0:45)

這頁要達成：把整場拉高到「這對貴公司有什麼用」。這是決定名次的一頁。

最後，我想回答一個問題：這套東西離開比賽還有沒有用？

左邊是我們這次交付的成果。但我想講右邊。

我們這個專案用的是這套流程的簡化版：任務卡就是 repo 裡的 Markdown 檔，靠 Git 留下歷程，因為我們只有兩個人。

〔停一拍〕

但在公司的多人環境裡，任務卡可以直接對應 GitLab 的 Issue，再由企劃、工程、設計、QA 各自審核自己關心的部分，接上既有的版控流程。

〔加重，這句慢〕

載體可以替換；任務的邊界、執行、審核與驗收這個閉環，不變。

我要誠實說明：我們這次沒有真的用 GitLab 跑過多人協作，這是我們認為可以擴展的方向，不是已經驗證的成果。

下一個專案不會複製《Meow準快跑》的玩法，但可以沿用這條生產線——把 AI 的速度，轉換成多人可以協作、可以追蹤的穩定交付。

Slide 20 | 致謝 (0:10)

這次交付的是《Meow準快跑》；真正留下來的，是下一款遊戲也能用的 AI 生產線。

遊戲和完整的製程都在螢幕上的連結裡，歡迎現場掃碼玩。謝謝各位。

〔停住，不要急著收電腦。等評審提問。〕

Q&A 預備

按被問到的機率排序。回答一律先給一句結論，再補一句依據。

1. 這算不算賭博？會不會有法規問題？

本作**只使用不可兌現的虛擬金幣，沒有充值、沒有提款、沒有現金獎勵**。我們借用的是風險決策的心理機制，不提供任何金流服務。簡報只陳述產品事實，不做跨司法管轄區的法律判斷。

2. 這樣算是你們做的，還是 AI 做的？

方向、決策、驗收是我們做的；實作大部分是 AI 做的。我們有二十六筆人類決策和十二次 AI 停下來提問的紀錄，可以清楚看到每一個轉折點是誰決定的。**我們的貢獻是把「怎麼讓 AI 穩定交付」這件事變成一套可重複的流程。**

3. 沒有 AI 的話要做多久？

誠實說我們沒有對照組。可以確定的是，這個規模的內容量——十關怪物、完整動畫、登入、雲端排行榜——兩個人用下班時間三週做完，沒有 AI 我們不會嘗試。**但省下來的不只是打字時間，更多是「不用等待彼此」**：工程和美術可以真正平行跑。

4. 遇到 AI 做錯的時候怎麼辦？

講 Slide 12 的 D-006 故事（版面盲改那段）。結論：**流程的價值是讓錯誤停在還原得回來的地方。**

5. 為什麼音效不用 AI？

十四個音效要的是精準和一致性，人工挑選比生成更快也更穩。我們沒有為了「全 AI」而硬用 AI——**選擇不用，也是一種判斷。**

6. 那個 agent-sprite-forge 是你們寫的嗎？

不是，那是 GitHub 上別人的開源 Codex skill，作者是 0x0funky，我們是使用者。放那頁是想說明這套「規格先行 + 確定性後處理」的思路，開源社群也在往同一個方向走。

7. 排行榜上那些名字是真的玩家嗎？

不是，有一份保底的模擬名單。因為比賽 demo 不能開天窗，所以正式版也會把這批 NPC 併進排行榜重新排名。這是我們主動揭露的設定，而且**不會寫假資料進資料庫**，只在前端合併。

8. 為什麼選 Godot 不選 Unity？

H5 導出輕、專案結構單純、純文字的場景檔對 AI 協作友善——AI 可以直接讀寫場景檔並用 Git diff 檢查。這對我們的流程很關鍵。

9. 這套流程導入公司要花多少成本？

最低成本的版本就是我們這次做的：一份 `AGENTS.md`、一個決策紀錄檔、一個提問檔，加上任務卡範本。**這些都是 Markdown，不需要買任何工具**。真正的成本是紀律——遇到模糊一定要停下來問，不能讓 AI 猜。

10. 三週是全職嗎？

不是，兩個人的下班時間。首個 commit 是 6 月 27 日，最後一張任務卡 7 月 16 日完成。

11. 為什麼是十關？倍率怎麼定的？

倍率盤在 `game_balance.json` 裡，從 1.25 倍到 38 倍。而且每一局會在基準上做隨機抖動、但強制單調遞增，**所以每一局的風險曲線都不一樣**。這些全部是資料，調整不用改程式。

12. 有做玩家測試嗎？

有做內部試玩並依回饋調整（危險度呈現、下注面板、結算的 FOMO 提示都改過），但**沒有做規模化的玩家測試**，這是我們的不足。

上台前檢查清單

內容 (今天做完)

- ☐ Slide 18 數字更新：27→28、25→26、120+→130+、5,500+→7,000+ (36 份→38 份)
- ☐ `ui_text.json` 的 `title_game_name` 改成「Meow準快跑」，重新 export H5 並部署
- ☐ 決定 Slide 8 留或砍 (建議砍，多 25 秒緩衝)
- ☐ 決定 Slide 12 的 D-006 加分段要不要講 (建議講)

Demo (彩排三次以上)

- ☐ 驗證 F4 → 挑戰的按鍵順序 (F4 是預備下一次，且無畫面回饋)
- ☐ demo 帳號在排行榜上已有名次，否則結算的排名提示是空的
- ☐ 餘額與下注額事先調好，讓收益級距看得出來
- ☐ 預先登入完成，停在下注畫面 (不要在台上等 OAuth)
- ☐ 預錄備援影片開在不用找的分頁
- ☐ 瀏覽器縮放、音量、投影比例確認 (BGM 切換是虎爺橋段的重點，音量一定要出得來)

講者

- ☐ 主講版計時 ≤ 13 分 15 秒
- ☐ 全程只說「虛擬金幣」，不說「賭」
- ☐ Slide 16 一定要說「這是別人的開源專案」
- ☐ Slide 19 一定要說「GitLab 是可擴展方向，不是已驗證成果」